DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Proportional Share Scheduling

CMSC 125 Problem Set 1

Prepared by: Rene Andre B. Jocsing

Published: February 16, 2026 **Deadline:** March 02, 2026

This problem set explores advanced scheduling concepts that extend beyond simple optimization metrics. You will investigate how modern operating systems implement fair-share scheduling to ensure processes receive CPU time proportional to their assigned priority or importance. This assignment requires research beyond the provided course materials to develop a comprehensive understanding of proportional-share scheduling mechanisms used in production systems.

Background

Traditional CPU schedulers like FCFS, SJF, STCF, and Round Robin focus on minimizing metrics such as turnaround time or response time. However, in multi-user systems, virtualized environments, and modern cloud computing platforms, a different goal emerges: ensuring that processes receive a *proportional share* of CPU time based on their assigned importance or priority. This paradigm shift introduces probabilistic and deterministic fair-share schedulers that fundamentally change how we think about resource allocation.

The concept of proportional-share scheduling addresses critical real-world requirements that optimization-based schedulers cannot satisfy. In a shared computing environment, how do we guarantee that a high-priority user receives twice the CPU time of a low-priority user? How do we prevent any single process from monopolizing system resources? These questions motivate the development of lottery scheduling, stride scheduling, and the Linux Completely Fair Scheduler (CFS).

Learning Objectives

After completing this problem set, students should be able to:

- Articulate the fundamental principles of proportional-share scheduling
- Explain the mechanics of lottery scheduling and its probabilistic guarantees
- Analyze stride scheduling as a deterministic alternative
- Understand the Linux CFS implementation and its design decisions
- Compare and contrast different fair-share scheduling approaches
- Evaluate the practical applicability of proportional-share schedulers
- Conduct independent research using academic sources
- Synthesize technical concepts into coherent written analysis

---

Essay Topic: Proportional Share Scheduling and Fair Resource Allocation

Using this OSTEP chapter on "Scheduling: Proportional Share" as your primary reference and additional academic sources, write a comprehensive essay that addresses the following four sections. Each section should constitute approximately 25% of your essay's content.

1. Proportional Share Fundamentals and Lottery Scheduling (25%)

Explain the core principle of proportional-share scheduling and how it differs philosophically from optimization-based schedulers (STCF, RR). Define the concept of **tickets** as the fundamental abstraction for representing resource share. Then, describe the **lottery scheduling** algorithm in detail:

- How does the lottery mechanism work? Walk through the process of selecting a winner from the ticket pool.
- Why is randomness advantageous in this context? Discuss the benefits mentioned in the OSTEP chapter (robustness to unexpected workloads, lightweight state management, computational efficiency) and provide concrete examples of scenarios where these advantages manifest.
- Using a specific example with three processes (Process A: 100 tickets, Process B: 50 tickets, Process C: 250 tickets), demonstrate how lottery scheduling achieves proportional allocation *probabilistically* over time. Show example lottery outcomes across multiple scheduling decisions and discuss how fairness emerges over longer time horizons. Calculate the expected CPU share for each process.

2. Advanced Lottery Mechanisms and Fairness Analysis (25%)

Lottery scheduling provides three powerful mechanisms for manipulating tickets: **ticket currency**, **ticket transfer**, and **ticket inflation**. For each mechanism:

- Explain how it works with a concrete example that illustrates the mechanism in action
- Describe a realistic use case where this mechanism would be beneficial (consider scenarios like user/kernel isolation, client-server interactions, or cooperative processes)
- Discuss any trust assumptions or security implications (e.g., what happens if a malicious process attempts to exploit the mechanism?)

Then, analyze the **fairness characteristics** of lottery scheduling. Using the fairness metric F = T_first / T_second discussed in the chapter (where perfect fairness equals 1.0), explain why lottery scheduling exhibits poor short-term fairness but converges to proportional fairness over longer time scales. What implications does this have for:

- Workloads with short-lived processes that complete in tens of milliseconds?
- Long-running compute-intensive processes that execute for hours?
- Interactive applications that require consistent response times?

3. Stride Scheduling and Deterministic Fair Share (25%)

Describe **stride scheduling** as a deterministic alternative to lottery scheduling. Your explanation should include:

- The concept of **stride** and **pass values**, including how they are computed from ticket allocations. Provide the mathematical formulas and explain the intuition behind them.
- The algorithm's decision-making process. Present clear pseudocode or a step-by-step description of how the scheduler selects the next process to run.
- A worked example using the same three processes from section 1 (Process A: 100 tickets, Process B: 50 tickets, Process C: 250 tickets). Show at least 10 scheduling cycles, displaying how stride and pass values evolve and demonstrating how stride scheduling achieves *exact* proportional sharing cycle by cycle.

Then, conduct a **critical comparison** between lottery and stride scheduling:

- What are the performance and correctness tradeoffs? Consider computational overhead, memory requirements, and determinism versus randomness.
- Why does lottery scheduling have an advantage regarding global state? Explain the problem stride

scheduling faces when a new process arrives mid-execution (hint: consider how pass values of existing processes relate to a new process with pass value 0).
- Under what circumstances would you choose lottery over stride, or vice versa? Provide specific workload characteristics or system requirements that would favor each approach.

4. Linux CFS and Real-World Proportional Scheduling (25%)

Explain how the **Linux Completely Fair Scheduler (CFS)** implements fair-share scheduling using a different approach from lottery or stride. Your discussion must include:

- The concept of **virtual runtime (vruntime)** and how it accumulates. How does vruntime differ from physical runtime? Why is this distinction important?
- The role of **sched_latency** (target latency for full scheduling cycle) and **min_granularity** (minimum time slice) parameters in balancing fairness and context-switching overhead. Explain the relationship between these parameters, the number of runnable processes, and the time slice calculation.
- How CFS uses process **nice values** (range: -20 to +19) and weights to implement priority. Show the weight calculation formula and explain how time slices are distributed proportionally based on weights. Provide a concrete example with processes at different nice values.
- Why CFS uses a **red-black tree** data structure instead of a simple list to organize runnable processes. What performance benefits does this provide at scale (consider O(log n) operations)? What is the tree's ordering key?
- How CFS handles **sleeping/waking processes** to prevent starvation while avoiding unfairness to CPU-bound processes. Discuss the problem of "sleep credit" and how CFS solves it.

Finally, reflect on the **practical applicability** of proportional-share schedulers:

- Discuss the "ticket assignment problem"—how does the system know how many tickets (or what nice value) to assign to different processes? Who decides? What information would be needed to make intelligent assignments?
- Why might proportional-share schedulers be particularly well-suited for virtualized environments and cloud computing? Consider scenarios with multiple virtual machines competing for physical CPU resources.
- What workload characteristics make proportional-share scheduling less effective than MLFQ or other general-purpose schedulers? When would you *not* want to use fair-share scheduling?

Research Requirements

This essay requires substantial research beyond the OSTEP textbook. You must:

- Cite **at least three (3) academically sound sources** using **APA 7th Edition** format
- The OSTEP textbook counts as one source; you need two additional sources minimum
- Acceptable sources include:
  - Peer-reviewed papers from conferences (ACM SIGOPS, USENIX, etc.)
  - Technical books on operating systems (see: syllabus)
  - Official Linux kernel documentation
  - Academic journal articles on scheduling algorithms, and etc.
- **Not acceptable**: Wikipedia, blogs, online forums, AI-generated content presented as original research
- Include a **References** section at the end following APA 7th Edition format
- The References section does **not** count toward your word count

Recommended research starting points:

- Original lottery scheduling paper: Waldspurger & Weihl (1994)
- Linux CFS design: Con Kolivas's work and Ingo Molnár's implementation
- Academic surveys on proportional-share scheduling
- Linux kernel documentation on the scheduler

Format Requirements

- Write on **one-whole sheets of yellow pad paper only**
- Work together with your group defined in this sheet to accomplish the essay
- Your essay must be approximately **1500 words** (acceptable range: 1350–1650 words)
- Essays outside this range will receive deductions
- **CRITICAL**: You must write the **cumulative word count at the left margin of each line**
  - Example: Line 1 ends at word 8, write "8" at the left margin
  - Line 2 ends at word 17, write "17" at the left margin
  - Continue throughout the entire essay
- Write **legibly in PRINT** (not cursive)
- Illegible answers will receive reduced credit
- **Minimal erasures**: If you make a mistake, neatly cross out with a **single line** (avoid correction fluid or tape!)
- Excessive corrections leading to illegible writing will result in deductions
- You may include hand-drawn diagrams, tables, or pseudocode to illustrate your points
- Such visual aids do not add to the essay's word count but can enhance clarity

---

Submission Instructions

1. Submission must occur **in-person during class hours** to the instructor on or before **March 2, 2026**
2. All yellow pad sheets must be **stapled together** securely in the upper-left corner
3. Write your **FULL NAMES** on the top left of the first page
4. Write the **TOTAL WORD COUNT** on the top right of the first page
5. Include page numbers on the bottom right of each sheet
6. As per the syllabus, late submissions for lecture requirements will **NOT** be accepted except with university-sanctioned documentation (medical emergencies, family emergencies, etc.)

Academic Honesty

This is a **paired assessment** (groups of 2, or groups of 3 if there is an odd number of students). Discussion with your peers is allowed to facilitate learning and deepen understanding, but the final output must come entirely from you. You may use AI tools (ChatGPT, Claude, Deepseek, etc.) for research and to aid your understanding of complex concepts, but you must read, comprehend, and synthesize the information yourself.

The final essay must be written by hand in your own words. As per the syllabus, direct copying from AI outputs or other sources is a serious academic offense. Violations will result in:

- Automatic failure in the course (5.0)
- Referral to the university disciplinary committee
- Possible expulsion from the university

Proper citation of all sources is mandatory. Plagiarism (presenting others' work as your own) will be prosecuted to the full extent of university policy.

Evaluation Criteria

Your essay will be evaluated on five dimensions:

- **Technical Accuracy (30%)**: Correctness of OS concepts, mechanism descriptions, and calculations
- **Depth of Analysis (25%)**: Synthesis of concepts, exploration of tradeoffs, critical thinking
- **Research Quality (20%)**: Quality and integration of academic sources, proper citations
- **Organization & Clarity (15%)**: Logical flow, clear explanations, precise technical writing
- **Format Compliance (10%)**: Adherence to word count, line numbering, legibility, and submission requirements

See the detailed rubric on the final page for specific performance standards.

Tips for Success

- **Start early**: Research and synthesis take time. Do not leave this for the last day.
- **Read the OSTEP chapter thoroughly**: This is your primary reference and foundation.
- **Find quality sources**: Use Google Scholar, IEEE Xplore, or ACM Digital Library.
- **Take notes while researching**: Organize information by section to avoid duplication.
- **Draft an outline**: Plan your argument structure before writing.
- **Work examples by hand**: Ensure you understand lottery and stride scheduling calculations.
- **Write clearly**: Explain concepts as if teaching someone unfamiliar with the topic.
- **Proofread**: Check for technical errors and writing clarity before final submission.
- **Practice line counting**: Do a test page to ensure you can maintain accurate cumulative counts.

Common Pitfalls to Avoid

- **Surface-level description**: Don't just explain *what* mechanisms do—analyze *why* they work and *when* they fail.
- **Missing examples**: Abstract descriptions without concrete examples are hard to follow and demonstrate weak understanding.
- **Ignoring tradeoffs**: Every scheduling algorithm has strengths and weaknesses. Acknowledge both.
- **Poor source integration**: Don't just list sources—use them to support your arguments.
- **Calculation errors**: Double-check all mathematical examples and scheduling simulations.
- **Format violations**: Forgetting line counts or exceeding word limits will cost you points.

Why Handwritten?

You may wonder why this assignment requires handwritten submission in an age of word processors and digital documents. This is a deliberate pedagogical decision designed to maximize your learning outcomes.

The AI Reality: Modern AI tools can generate technically accurate essays on operating systems concepts. However, this course aims to develop *your* understanding, not the AI's. The handwritten requirement creates a necessary friction point in the workflow.

Forced Engagement: If you use AI assistance for research or drafting, you cannot simply copy-paste the output. You must physically transcribe every word by hand. This process forces you to:

- **Read every sentence carefully**: You cannot transcribe text you haven't read
- **Process the information**: Handwriting is slower than typing, giving your brain time to process concepts as you write them
- **Identify gaps in understanding**: When you write something you don't understand, it becomes immediately apparent. You'll naturally pause, reread, and seek clarification
- **Engage with technical details**: Transcribing formulas, examples, and calculations requires active attention to detail

Cognitive Benefits: Research in educational psychology consistently demonstrates that handwriting promotes deeper cognitive processing than typing (this is well-known). The act of forming letters, combined with the slower pace, enhances memory encoding and conceptual understanding. When you handwrite technical content, you're more likely to remember it during examinations and apply it in future coursework.

Academic Integrity: This format also serves an integrity function. While AI assistance is permitted for research and understanding, the final product must demonstrate *your* comprehension. A handwritten essay in your own words ensures the work is authentically yours. We can distinguish between genuine understanding and superficial reproduction.

Professional Preparation: In technical interviews and graduate school qualifying exams, you will often work through problems by hand on whiteboards or paper. This assignment provides practice in organizing complex technical arguments without digital aid—a valuable professional skill.

The goal is not to make your life difficult, but to ensure that the time you invest in this assignment translates into genuine learning. When you sit for future examinations, beyond the walls of the university, the concepts you've handwritten will be far more accessible in your memory than those you merely copy-pasted into a document.

Grading Rubric

Criteria	Excellent (90–100%)	Good (75–89%)	Fair (60–74%)	Poor (0–59%)
Technical Accuracy (30%)	All OS concepts correct; mechanisms described precisely; calculations shown correctly; examples accurate; no conceptual errors.	Core concepts correct; minor inaccuracies in details; calculation errors or missing steps; examples mostly accurate.	Some correct concepts; significant errors in mechanisms; failed calculations; confused examples.	Fundamental misunderstanding; incorrect mechanisms throughout; no accurate calculations or examples.
Depth of Analysis (25%)	Synthesizes concepts across sections; explores tradeoffs deeply; concrete examples throughout; anticipates edge cases; critical thinking evident; goes beyond surface description.	Good analysis; explores some tradeoffs; provides adequate examples; shows solid understanding of implications and relationships.	Mostly descriptive; limited analysis; few examples; surface-level understanding; minimal exploration of tradeoffs.	Purely descriptive; no analysis; regurgitates definitions; fails to address prompt requirements; no critical thinking.
Research Quality (20%)	3+ high-quality academic sources; meaningful integration into argument; perfect APA citations; sources directly support claims; demonstrates independent research.	3 acceptable sources; generally correct APA format; relevant sources with adequate integration; some independent research evident.	Minimum sources met; citation errors; weak integration; sources tangentially relevant or poorly utilized.	Missing required citations; non-academic sources only; no APA format; plagiarism detected; no evidence of research.
Organization & Clarity (15%)	Logical flow between sections; clear topic sentences; smooth transitions; precise technical writing; arguments build coherently; easy to follow throughout.	Generally organized; mostly clear explanations; adequate transitions; readable throughout with minor structural issues.	Weak organization; unclear explanations; choppy transitions; difficult to follow in places; some technical writing issues.	Incoherent structure; incomprehensible explanations; no clear argument thread; impossible to follow.
Format Compliance (10%)	Word count within range; cumulative line counts correct throughout; legible print handwriting; minimal/no erasures; proper References section; all requirements met.	Minor word count deviation (<50 words); line counts mostly correct with few gaps; mostly legible; some erasures but readable; References present with minor errors.	Significant word count deviation (50-100 words); many missing line counts; partially illegible; excessive erasures; incomplete References.	Word count requirements ignored; missing line counts; illegible handwriting; violates format requirements; no References; critical submission violations.